

Design specification:

Binary files:

- still via streams but with binary flag specified, the character interpretation is removed, data is moved to RAM as it is
- Instead of using normal reading/writing use `.read()` `.write()` operations which are operating with bytes

Theoretical performance analysis:

Vectors are sized at the start using the `vector<double>(M * M)` constructor, avoiding expensive dynamic reallocations during processing.

Implementation specification:

All the matrix are stored by the program using vectors, so that it is more efficiently allocated the memory.

Using the no of threads specified in the program args, I created the transpose for the matrix B using the no of threads and multiplied $A \times B$ by allocating to each thread a block of rows for the matrix A to multiply with all the lines of the matrix B

Testing:

Because the data is written as bytes in binary files, the reading and writing is much easier

Multiplying in a parallel mode, it's more efficient than sequential multiplying, and we can see the time difference between using 20 threads and 40 threads

COMPLEXITY:

La cerinta 3 avem timpi putin mai mari decat la cerinta 2 deoarece programul atunci cand intalneste `#pragma omp parallel` OpenMP trebuie sa verifice starea poolului, sa trezeasca thread-urile "adormite" si sa le asigneze sarcini.

Acest proces adauga un mic overhead timpului de executie fata de modul in care am 1 assignat manual la threaduri sarcinile in cerinta 2.

`#pragma omp parallel for` introduce automat o bariera, asteapta ca toate threadurile sa ajunga aici

3 A) CITIRE: $O(M^2)$ TRANSPUSA $O(M^2/NO_THREADS)$ MULTIPLICARE
 $O(M^3/NO_THREADS)$ SCRIERE $O(M^2)$

3 B) CITIRE: $O(M^2/NO_THREADS)$ TRANSPUSA $O(M^2/NO_THREADS)$ MULTIPLICARE $O(M^3/NO_THREADS)$ SCRIERE $O(M^2)$

Test cases:

- 1) M=1000
- 2) M=100000

10 executions => average corresponding times

M=1000

Var	T_reading	T_addition	T_writing	T_total	Speed-up (T_seq/T_par)
1.a	222.30 ms	2948.30 ms	279.40 ms	3466.40 ms	
1.b	3.30 ms	2950.40 ms	5.80 ms	2973.10 ms	
2.a	20th: 2.20 ms 40th: 2.00 ms	20th: 441.10 ms 40th: 398.30 ms	20th: 7.20 ms 40th: 7.40 ms	20th: 455.00 ms 40th: 411.30 ms	20th: 6,534 40th: 7,228
2.b	20th: 1.30 ms 40th: 2.20 ms	20th: 412.30 ms 40th: 379.40 ms	20th: 9.10 ms 40th: 7.60 ms	20th: 427.10 ms 40th: 393.30 ms	20th: 6,959 40th: 7,559
3.a	20th: 2.90 ms 40th: 2.70 ms	20th: 577.60 ms 40th: 520.60 ms	20th: 8.70 ms 40th: 7.80 ms	20th: 589.90 ms 40th: 531.90 ms	20th: 5,04 40th: 5,589

3.b	20th: 1.50 ms 40th: 2.20 ms	20th: 503.00 ms 40th:513.50 ms	20th: 8.90 ms 40th: 9.00 ms	20th: 514.40 ms 40th:525.80 ms	20th:5,779 40th:5,654
4.a					
4.b					
5.a					
5.b					
6					
7.a					
7.b					

M=100000 TO BE DONE ON THE FACULTY CLUSTER – CANNOT DO IT LOCALLY

Var	T_reading	T_addition	T_writing	T_total	Speed-up (T_seq/T_par)
1.a					
1.b					
2.q					
2.b					
3.a					
3.b					
4.a					
4.b					
5.a					
5.b					
6					
7.a					
7.b					